

Task1_QA_Report_Sunil

Sunil Varma Tenali

May 20, 2026

Application Tested: React Redux RealWorld Example App

Repository: <https://github.com/gothinkster/react-redux-realworld-example-app>

Local URL: <http://localhost:4100>

Browser: Google Chrome

Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	App crashes on startup due to unsafe error handling	Run <code>npm start</code> and open <code>http://localhost:4100</code> .	Expected: The homepage should load normally. Actual: The app shows <code>Unhandled Rejection (TypeError: Cannot read properties of undefined (reading 'body'))</code> .	Critical	The middleware assumes <code>error.response.body</code> always exists. When the request fails without a valid response object, the app crashes while handling the error.
2	Signup fails because API request is blocked by CORS/API redirect	Open the app locally, go to signup, enter valid signup details, and submit the form.	Expected: The user account should be created and the user should be logged in. Actual: The request to <code>/api/users</code> is blocked by CORS, and the app crashes.	High	The app uses an API endpoint that redirects from <code>conduit.productionready.io</code> to <code>api.realworld.io</code> . The redirected API does not allow requests from the local frontend origin.
3	Login fails because API request is blocked by CORS/API redirect	Open the app locally, go to login, enter email and password, and submit the form.	Expected: The user should either log in successfully or see a clear invalid credentials message. Actual: The login request is blocked by CORS and the app crashes.	High	The external API does not allow requests from <code>http://localhost:4100</code> . The frontend middleware also does not handle failed network responses safely.
4	Global Feed fails to load due to API/CORS/HTTP 530 issue	Start the app locally, close the React error overlay, and click Global Feed .	Expected: Public articles should load, or the app should show a clear fallback message. Actual: The request to <code>/api/articles?limit=10&offset=0</code> fails with HTTP 530/CORS error and the app crashes.	High	The feed depends on an external API that fails for local requests. The UI does not gracefully handle failed article loading.
5	Signup form submits empty/invalid data without client-side validation	Open <code>/register</code> , leave fields empty or invalid, and click Sign up .	Expected: The form should block submission and show field-level validation messages before making an API request. Actual: The app sends a POST request to <code>/api/users</code> , the request fails, and the app crashes.	Medium	The frontend does not validate required fields before submission and relies on the backend/API to reject invalid input.

Root-Cause Analysis

- The most significant issue is the application crash caused by unsafe error handling in the middleware. The application attempts to access `error.response.body` whenever an API request fails. This works only when the failed request returns a normal structured HTTP response. However, in this test environment, multiple requests fail because of CORS restrictions, redirects, or network-level failures. In those cases, the error object does not contain a valid `response` property.
- Because the middleware does not check whether `error.response` exists before reading `body`, the application throws a second error while trying to handle the original failed request. This causes the React development error overlay to appear and blocks the user from continuing normally. The user impact is high because core flows such as signup, login, and Global Feed loading become unusable.
- To fix this, I would update the middleware to safely check whether `error.response` exists before accessing nested properties. I would also add a fallback error message such as “Service unavailable, please try again later.” The API base URL should also be reviewed so local development does not depend on an endpoint that redirects to a CORS-blocked domain. Finally, regression tests should be added for network failures, CORS failures, and API responses without a body.